

[Home](#) > [Blog](#) > [Data Engineering](#)

Database Sharding: Examples, Strategies, Tools, and More

Learn what database sharding is, how it works, how it differs from partitioning and replication, and what strategies you can use for sharding.

Jun 4, 2025 · 7 min read



Marie Fayard
Tech Lead & Product Strategist

TOPICS

Data Engineering

I had been a full-stack engineer for a couple of years when I was assigned my first data engineering project. Up to that point, I had coded on the frontend, developed APIs, and managed and deployed cloud infrastructure pretty regularly.

Like many engineers in early-stage startups, I had also touched databases, designing a new table or schema here and there, and migrating small amounts of data around. But then, the company I worked for started gathering huge amounts of data, and the database performance started to drop.

Our applications were slowing down, latency was increasing, and we were even experiencing downtime during peak loads. It was a very frustrating problem that my team worked hard to fix. That's when I learned a ton about databases, how they work, and how to optimize them.

In this blog, I'll explain one of the concepts that really made a difference then and has done many more times over the last few years: **database sharding**.

What Is Database Sharding?

Database sharding is the practice of splitting a database into smaller subsets, or *shards*, with each shard functioning as its own independent database. Think of it like breaking up one giant spreadsheet into several smaller, more manageable ones, each handling its own share of the workload.

Why and when to shard a database?

Sharding is NOT the answer to every database problem.

There are other horizontal scaling methods, like replication or vertical scaling methods, that are much easier to implement. Have a look at these first before jumping into sharding!

Sharding comes into play when your database starts struggling to keep up with the demands of a growing application. Maybe your user base has exploded, or your application is generating huge amounts of data every second. If you're noticing slow queries, increased latency, or frequent downtime during peak traffic, it's probably a sign that your database is hitting its limits.

One big reason for sharding is scalability. As your data grows, adding more hardware to a single database server (vertical scaling) only gets you so far, and it can get very expensive. Sharding, on the other hand, allows you to scale horizontally by distributing data across multiple servers, each handling a portion of the load.

Another reason to choose sharding is performance. Even if your database can handle the data volume, querying through massive amounts of information can take time. By sharding, you're working with smaller, more focused datasets, which means faster queries and reduced contention for resources.

As for when to shard, it's usually best to consider it when your application is starting to outgrow what a single database can handle. The exact timing depends on a lot of variables, like your resources and predicted growth, and is a bit of a fine line to walk. Implementing sharding too early can introduce unnecessary complexity when a single database or simple optimizations would have been enough, but waiting too long can make the transition painful.

A good rule of thumb is to monitor your database performance metrics closely and plan ahead. If you're noticing bottlenecks or rapid data growth that could cause issues in the near future, it's probably time to explore sharding. Trust me, getting ahead of the problem will save you a lot of headaches down the road!

Database Sharding vs. Partitioning

At first glance, sharding and partitioning may seem like the same concept because they both involve splitting data into smaller pieces for better organization and performance. But the two serve different purposes and are suited for different scales of operation.

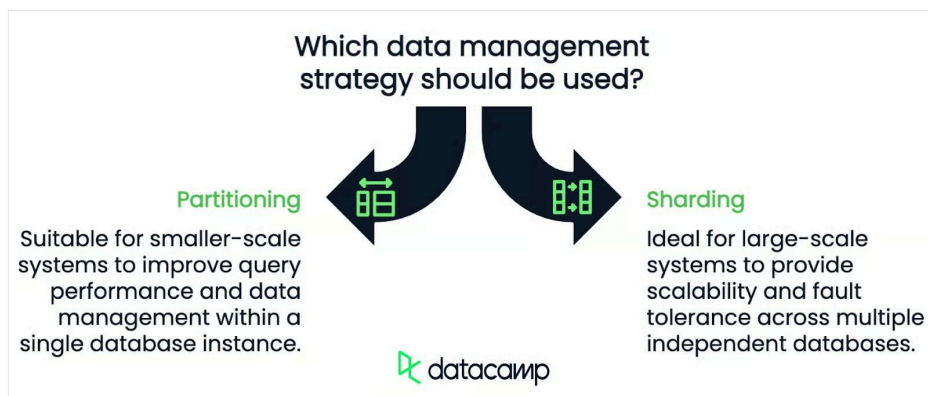
Partitioning is the process of dividing data into logical segments within the same database instance. These segments, often called partitions, can be based on criteria like range, list, or hash. For example, in a user database, you might store users born in January in one partition, February in another, all within the same database.

Partitioning is an excellent choice for smaller-scale systems that don't yet need to scale horizontally. It improves query performance by limiting the amount of data scanned during queries and makes data management more efficient.

Sharding, on the other hand, takes partitioning to the next level by distributing these logical segments across multiple independent databases. Each shard is a standalone database that handles its own data and queries.

This approach is ideal for large-scale systems where a single database can't handle the load anymore. Sharding provides both scalability and fault tolerance since each shard operates independently.

From my experience, partitioning is often a stepping stone to sharding. If you're starting to see performance issues but your database isn't maxed out yet, partitioning can buy you time while you prepare for a larger-scale solution like sharding.



If you are interested in database design, take a look at this [full database design course](#). You'll learn how to organize, manage, and optimize SQL databases, and even how to set up different types of partitioning.

Database Sharding vs. Replication

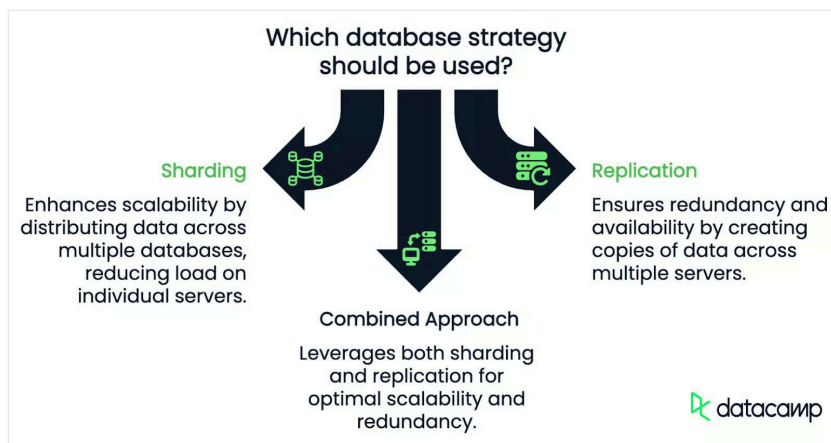
Similarly to the above, sharding and replication are often discussed side by side, but they address different problems and have distinct use cases.

The primary goal of sharding is scalability. By dividing the data and workload across multiple databases, you reduce the load on any single database. Sharding ensures that as your data grows, you can add more shards to accommodate it, maintaining performance and reducing the risk of overloading a single database.

Replication, on the other hand, involves creating copies of the same data across multiple servers. Every replica contains the entire dataset and can independently handle read operations, which means that replication is focused on redundancy and availability rather than scalability.

By having multiple copies of your database, you protect against data loss and ensure that your system stays online even if one server goes down, but it doesn't solve the problem of growing data volumes. In fact, since every replica stores the full dataset, storage requirements grow proportionally with the number of replicas. On top of that, *write* operations can become a bottleneck, since they must be applied to all replicas to keep the data consistent.

The thing is, it's not necessarily one or the other. In practice, many large-scale systems use both sharding and replication together. For instance, a sharded database might also have replicas of each shard.



How Does Database Sharding Work?

Okay, so how exactly does sharding work? Let's break it down into three key components.

Partitioning logic

The first step in sharding is figuring out how to divide your data. Your partitioning logic is the set of rules that determines how data is split across shards. There are several common ways to do this:

- **Range-based partitioning:** Data is divided based on a range of values, like user IDs or timestamps. For example, users with IDs 1-1000 go to one shard, while 1001-2000 go to another. This approach is simple but can lead to uneven distribution if your data isn't spread out evenly.
- **Hash-based partitioning:** Data is assigned to shards based on the output of a hash function. This method is great for evenly distributing data but can make rebalancing tricky if you need to add or remove shards.
- **Geographic partitioning:** Data is divided based on a user's location, such as assigning all European users to one shard and all North American users to another. This approach is especially useful for reducing latency in location-based applications.

The choice of partitioning logic depends on your data and application needs. I've personally seen range-based partitioning work well for chronological data, like transaction histories,

while hash-based methods have worked better in systems with unpredictable workloads.

Shard mapping

Once your data is partitioned, you need a way to map each partition to its corresponding shard.

Some systems use static mapping, where data is assigned to shards based on fixed rules. Others use dynamic mapping, often managed by a central metadata service, which makes it easier to add or remove shards as needed. Either way, the goal is to ensure that your system knows exactly where to find any given piece of data.

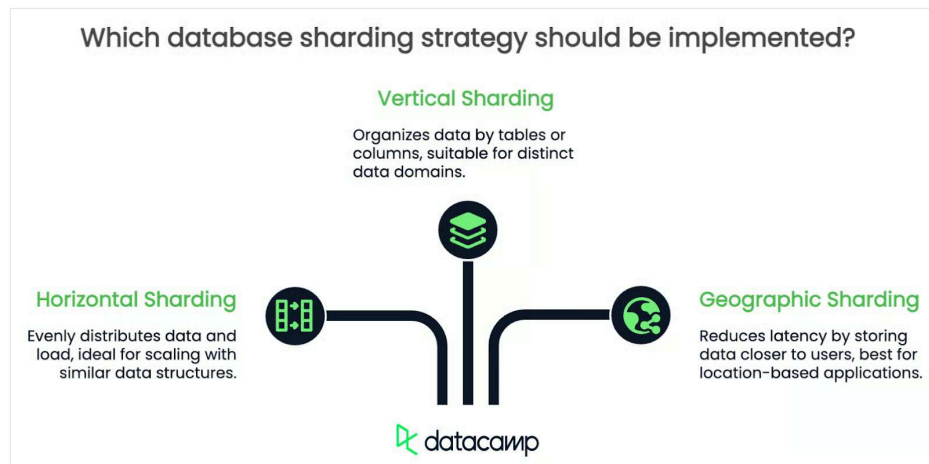
Query routing

Finally, you need a mechanism to ensure that queries are sent to the correct shard. This is called query routing, and it's important for making sharding work for your application. When an application sends a query, the routing logic determines which shard (or shards) hold the relevant data and sends the request there.

In some cases, this can be as simple as the application itself knowing where to send each query based on the partitioning logic. In other setups, a proxy or middleware layer handles routing, abstracting the complexity away from the application. I've worked with both approaches, and while proxies add some overhead, they can save you a lot of effort in larger systems.

Database Sharding Strategies

There is no single approach to sharding. The strategy you choose very much depends on the structure of your data and how you use it. I'll go over some of the most common ones, but there are many more than three.



Horizontal sharding

Horizontal sharding splits rows of a table across multiple shards. For example, if you're managing a user database, you might split users into shards based on their ID: users 1-1000 in one shard, 1001-2000 in another, and so on.

This approach is one of the most popular because it evenly distributes data and load, making it easier to scale horizontally as your application grows. It's especially effective when your tables have lots of rows with similar structures.

When using horizontal sharding, you need to be very careful when you choose your partitioning logic. Poorly distributed data means drastically uneven workloads, and some shards will be overloaded while others sit idle.

Vertical sharding

In vertical sharding, you split your database by tables or columns. For instance, you might put user profiles in one shard, transaction data in another, and logs in a third. Each shard is essentially responsible for a specific subset of data.

This strategy works well when your application has clear, distinct data domains that don't interact much, but one thing to watch out for is cross-shard queries. If your application frequently needs to combine data from different shards, vertical sharding can actually create more performance bottlenecks than you had to start with!

Geographic sharding

Geographic sharding organizes data based on the user's location. For example, all European user data might live in one shard, while North American data resides in another. This strategy reduces latency by ensuring that users' data is stored closer to where they're accessing it.

However, geographic sharding comes with its own challenges. You'll need to carefully handle users who travel between regions or whose data spans multiple locations. Keeping data consistent across shards can also get tricky.

Sometimes, combining multiple strategies can be the best solution—for example, using horizontal sharding within geographic shards. The key is to balance simplicity, scalability, and performance. You'll have to maintain your database after you create these shards, so make it easy for your future self!

Tools and Frameworks for Database Sharding

You don't have to start from scratch and implement sharding all by yourself. There are many tools and frameworks that make sharding easier and offer built-in features to handle the complexity for you.

MySQL with ProxySQL

[ProxySQL](#) acts as a smart proxy layer and handles query routing and load balancing for sharded [MySQL](#) databases. It maintains a mapping of which shard holds what data, and, when a query comes in, it automatically routes it to the appropriate shard so your application doesn't have to handle the logic.

I have never worked with ProxySQL myself, but a friend of mine testified that its ability to handle query routing saved him hours of manual configuration!

MongoDB

[MongoDB](#) has built-in sharding support, which makes it a popular option for handling large unstructured datasets. MongoDB uses a sharding key to determine how documents are



EN

[Buy Now >](#)

PostgreSQL with Citus

Citus is a great option for those who prefer relational databases. Citus is essentially an extension that transforms [PostgreSQL](#) into a distributed database.

I've worked in a team that used Citus to scale a SaaS application, and it allowed us to maintain the robustness of PostgreSQL and structured tables while handling massive amounts of data.

Citus uses a distributed query planner to handle complex queries across shards, and it's especially useful for analytics-heavy applications where you need to combine data from multiple shards. The learning curve was steeper than MongoDB's built-in sharding, but the results were worth it!

Conclusion

I hope this guide has given you the tools to explore sharding in your own systems (or the confidence to say that it isn't the right solution for you right now!). Remember, it's not a one-size-fits-all solution, but when used thoughtfully, it's a powerful tool for creating scalable and efficient architectures.

And as someone who learned these lessons the hard way, my advice is this: don't wait until your database is on its knees to start thinking about sharding. Plan ahead, experiment, and adapt! Your future self (and your users) will thank you.

FAQs

Can I implement sharding incrementally, or does it require a full redesign? ^

While it's easier to plan for sharding during the initial database design, it's possible to implement it incrementally. You can start by sharding only specific tables that are causing performance issues or by routing new data to shards while leaving existing data as-is.

What happens if a shard fails? v

Do all shards need to be the same size? v

Can I still run joins across shards? v

How does sharding impact backups? v



AUTHOR

Marie Fayard

I am a product-minded tech lead who specialises in growing early-stage startups from first prototype to product-market fit and beyond. I am endlessly curious about how people use technology, and I love working closely with founders and cross-functional teams to bring bold ideas to life. When I'm not building products, I'm chasing inspiration in new corners of the world or blowing off steam at the yoga studio.

TOPICS

Data Engineering



👥 Training more people?

Get your team access to the full DataCamp for business platform.

For Business

For a bespoke solution [book a demo](#).

Learn data engineering with these courses!

Professional Data Engineer in Python

🕒 40 hr

Dive deep into advanced skills and state-of-the-art tools revolutionizing data engineering roles today with our Professional Data Engineer track.

[See Details →](#)

[Start Course](#)

[See More →](#)

Related



BLOG

Sharding vs Partitioning:
Understanding Database...



BLOG

What Is Data Partitioning? A
Complete Guide for Beginners



BLOG

Time Series Database (TSDB): A
Guide With Examples

[See More →](#)

Grow your data skills with DataCamp for Mobile

Make progress on the go with our mobile courses and daily 5-minute coding challenges.



LEARN

[Learn Python](#)

[Learn AI](#)

[Learn Power BI](#)

[Learn Data Engineering](#)

[Assessments](#)

[Career Tracks](#)

[Skill Tracks](#)

[Courses](#)

Data Science Roadmap

DATA COURSES

Python Courses

R Courses

SQL Courses

Power BI Courses

Tableau Courses

Alteryx Courses

Azure Courses

AWS Courses

Google Sheets Courses

Excel Courses

AI Courses

Data Analysis Courses

Data Visualization Courses

Machine Learning Courses

Data Engineering Courses

Probability & Statistics Courses

DATALAB

Get Started

Pricing

Security

Documentation

CERTIFICATION

Certifications

Data Scientist

Data Analyst

Data Engineer

SQL Associate

Power BI Data Analyst

Tableau Certified Data Analyst

Azure Fundamentals

AI Fundamentals

RESOURCES

[Resource Center](#)

[Upcoming Events](#)

[Blog](#)

[Code-Alongs](#)

[Tutorials](#)

[Docs](#)

[Open Source](#)

[RDocumentation](#)

[Book a Demo with DataCamp for Business](#)

[Data Portfolio](#)

PLANS

[Pricing](#)

[For Students](#)

[For Business](#)

[For Universities](#)

[Discounts, Promos & Sales](#)

[Expense DataCamp](#)

[DataCamp Donates](#)

FOR BUSINESS

[Business Pricing](#)

[Teams Plan](#)

[Data & AI Unlimited Plan](#)

[Customer Stories](#)

[Partner Program](#)

ABOUT

[About Us](#)

[Learner Stories](#)

[Careers](#)

[Become an Instructor](#)

[Press](#)

[Leadership](#)

[Contact Us](#)

[DataCamp Español](#)

[DataCamp Português](#)

[DataCamp Deutsch](#)

[DataCamp Français](#)

SUPPORT

[Help Center](#)

[Become an Affiliate](#)



[Privacy Policy](#)

[Cookie Notice](#)

[Do Not Sell My Personal Information](#)

[Accessibility](#)

[Security](#)

[Terms of Use](#)

© 2026 DataCamp, Inc. All Rights Reserved.